



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №6

із дисципліни «Технології розроблення програмного забезпечення»

Тема: «Патерни проектування»

Тема проекту: «Музичний програвач»

GitHub: <https://github.com/martyshenkoo/MusicPlayerWeb>

Виконала:

студентка групи ІА-31:

Мартишенко І.С.

Перевірів:

Мягкий М.Ю.

Мета: Вивчити структуру шаблонів «Abstract Factory», «Factory Method», «Memento», «Observer», «Decorator» та навчитися застосовувати їх в реалізації програмної системи.

Музичний програвач (iterator, command, memento, facade, visitor, client-server)

Музичний програвач становить собою програму для програвання музичних файлів або відтворення потокової музики з можливістю створення, запам'ятовування і редагування списків програвання, перемішування/повторення (shuffle/repeat), розпізнавання різних аудіо-форматів, еквалайзер.

Завдання

- Ознайомитись з короткими теоретичними відомостями.
- Реалізувати частину функціоналу робочої програми у вигляді класів та їхньої взаємодії для досягнення конкретних функціональних можливостей.
- Реалізувати один з розглянутих шаблонів за обраною темою.
- Реалізувати не менше 3-х класів відповідно до обраної теми.
- Підготувати звіт щодо виконання лабораторної роботи. Поданий звіт повинен містити: діаграму класів, яка представляє використання шаблону в реалізації системи, навести фрагменти коду по реалізації цього шаблону.

Теоретичні відомості

Призначення: Шаблон використовується для збереження і відновлення стану об'єктів без порушення інкапсуляції. Об'єкт «Memento» служить виключно для збереження змін над початковим об'єктом (Originator). Лише початковий об'єкт має можливість зберігати і отримувати стан об'єкту «Memento» для власних цілей, цей об'єкт є «порожнім» для кого-небудь ще. Об'єкт «Caretaker» використовується для передачі і зберігання мemento об'єктів в системі.

Таким чином вдається досягти наступних цілей:

- зберігання стану повністю відділяється від початкових об'єктів, що полегшує їх реалізацію;
- передача об'єктів «Memento» лягає на плечі Caretaker об'єктів, що

дозволяє гнучкіше управляти станами об'єктів і спростити дизайн класів початкових об'єктів;

- збереження і відновлення стану реалізовані у вигляді двох простих методів і є закритими для кого-небудь ще окрім початкових об'єктів, таким чином не порушуючи інкапсуляцію.

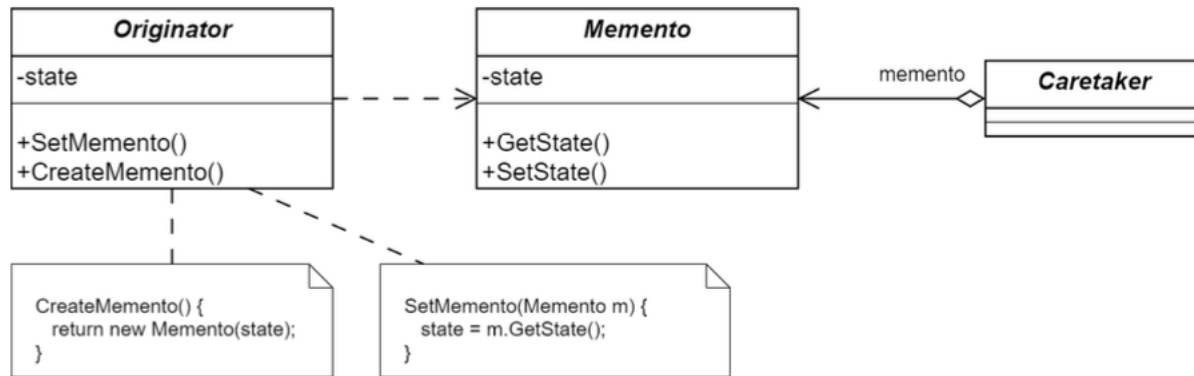


Рис-1. Структура шабону «Знімок»

Шаблон «Мементо» дуже зручно використати разом з шаблоном «Команда» для реалізації «скасовних» дій – дані про дію зберігаються в мементо, а команда має можливість вважати і відновити початкове положення відповідних об'єктів.

Переваги та недоліки:

- + Не порушує інкапсуляцію вихідного об'єкта.
- + Спрощує структуру вихідного об'єкта. Не потрібно зберігати історію версій свого стану.
- Вимагає багато пам'яті, якщо клієнти дуже часто створюють знімки.
- Може спричинити додаткові витрати пам'яті, якщо об'єкти, що зберігають історію, не звільняють ресурси, зайняті застарілими знімками.

У музичному програвачі, де користувач може створювати плейлісти, перейменовувати їх, додавати або видаляти треки, часто виникає потреба повернути попередній стан, наприклад, відмінити видалення трека або повернути стару назву плейліста. Саме тому шаблон Мементо є доречним у даному проєкті.

Патерн дозволяє зберігати стан вибраного об'єкта (плейліста або списку треків) перед виконанням операції та відновлювати його у випадку, якщо дія була помилковою або небажаною. У поєднанні з шаблоном Команда, який інкапсулює операції над

плейлістами та треками, «Мементо» дає можливість реалізувати механізм «Undo» без порушення інкапсуляції.

Хід роботи

У межах проєкту «Музичний програвач» було реалізовано основну логіку роботи з користувачами та аудіотреками.

Реалізувати частину функціоналу робочої програми у вигляді класів та їхньої взаємодії для досягнення конкретних функціональних можливостей.

```
Services > C# PlayerStateService.cs
1  using MusicPlayerWeb.Models;
2
3  namespace MusicPlayerWeb.Services;
4
5  public class PlayerStateService
6  {
7      private readonly object _sync = new();
8      private PlayerStateSnapshot _snapshot = PlayerStateSnapshot.Empty;
9
10     public PlayerStateSnapshot Snapshot
11     {
12         get
13         {
14             lock (_sync)
15             {
16                 return _snapshot;
17             }
18         }
19     }
20
21     public void Play(string title, string url)
22     {
23         lock (_sync)
24         {
25             _snapshot = PlayerStateSnapshot.Create(title, url, PlayerPlaybackState.Playing);
26         }
27     }
28
29     public void Pause()
30     {
31         lock (_sync)
32         {
33             if (!_snapshot.HasTrack)
34                 return;
35
36             _snapshot = PlayerStateSnapshot.Create(_snapshot.Title, _snapshot.Url, PlayerPlaybackState.Paused);
37         }
38     }
39
40     public void Stop()
41     {
42         lock (_sync)
43         {
44             _snapshot = PlayerStateSnapshot.Create(null, null, PlayerPlaybackState.Stopped);
45         }
46     }
47 }
48
```

Рис-2. Services-PlayerStateService.cs

Models > C# PlayerStateSnapshot.cs

```
1 namespace MusicPlayerWeb.Models;
2
3 public class PlayerStateSnapshot
4 {
5     public string? Title { get; init; }
6     public string? Url { get; init; }
7     public PlayerPlaybackState State { get; init; }
8     public DateTime UpdatedAtUtc { get; init; }
9
10    public bool HasTrack => !string.IsNullOrEmpty(Title) && !string.IsNullOrEmpty(Url);
11
12    public static PlayerStateSnapshot Create(string? title, string? url, PlayerPlaybackState state)
13    {
14        return new PlayerStateSnapshot
15        {
16            Title = title,
17            Url = url,
18            State = state,
19            UpdatedAtUtc = DateTime.UtcNow
20        };
21    }
22
23    public static PlayerStateSnapshot Empty => Create(null, null, PlayerPlaybackState.Stopped);
24 }
25
```

Рис-3. Models-PlayerStateSnapshot.cs

```

Services > Commands > C# PlayerCommandInvoker.cs
1  using MusicPlayerWeb.Models;
2
3  namespace MusicPlayerWeb.Services.Commands;
4
5  public class PlayerCommandInvoker
6  {
7      private const int MaxHistory = 20;
8      private readonly List<PlayerCommandHistoryItem> _history = new();
9
10     public IReadOnlyCollection<PlayerCommandHistoryItem> History
11     {
12         get
13         {
14             lock (_history)
15             {
16                 return _history.ToArray();
17             }
18         }
19     }
20
21     public void Execute(IPlayerCommand command, string username)
22     {
23         command.Execute();
24         AddToHistory(command.Name, username);
25     }
26
27     private void AddToHistory(string commandName, string username)
28     {
29         lock (_history)
30         {
31             _history.Add(new PlayerCommandHistoryItem
32             {
33                 CommandName = commandName,
34                 ExecutedBy = username,
35                 ExecutedAtUtc = DateTime.UtcNow
36             });
37
38             if (_history.Count > MaxHistory)
39             {
40                 _history.RemoveRange(0, _history.Count - MaxHistory);
41             }
42         }
43     }
44 }
45

```

Рис-4. Services-Commands-PlayerCommandInvoker.cs

Реалізувати один з розглянутих шаблонів за обраною темою.

Для реалізації механізму збереження та відновлення стану музичного програвача було використано патерн Memento.

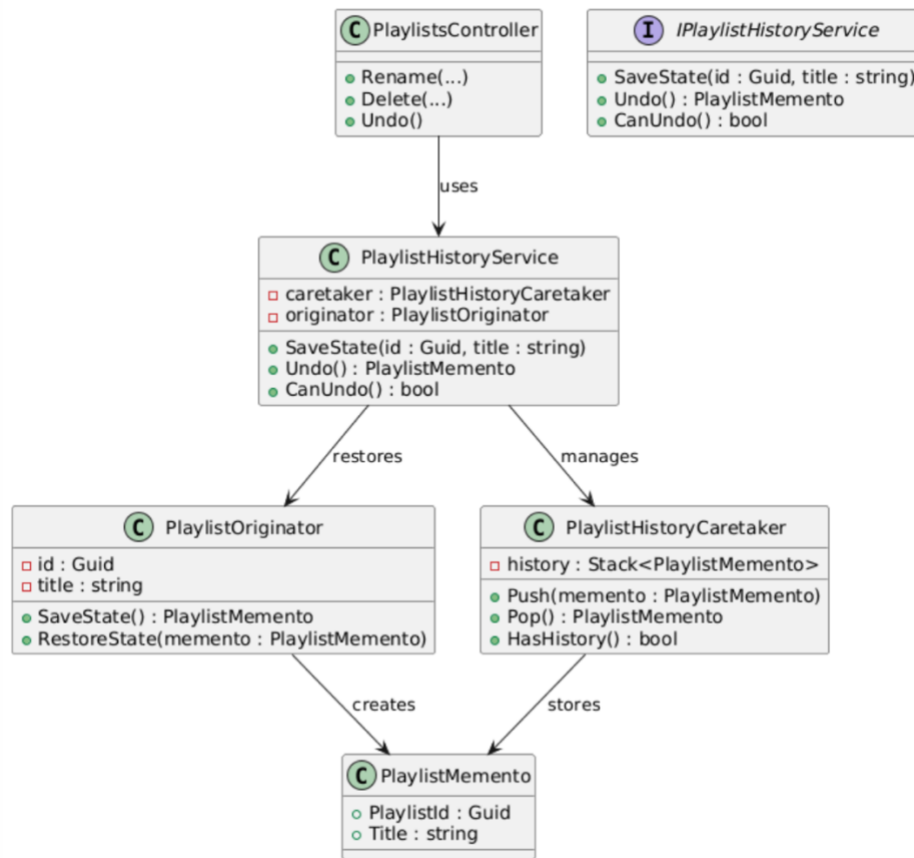


Рис-5. Структура патерну Memento

1. PlaylistMemento - об'єкт-знімок стану

`PlaylistMemento` зберігає попередній стан плейліста, включно з його назвою та ідентифікатором. Цей клас містить лише дані, необхідні для відновлення. `Memento` дозволяє зберігати стан без доступу до внутрішньої структури плейліста.

2. PlaylistOriginator - джерело стану

`PlaylistOriginator` створює `Memento` та відновлює стан плейліста на його основі.

Саме він знає, які дані потрібно зберегти перед операціями зміни плейліста.

`Originator` використовується при виконанні `Undo`.

3. PlaylistHistoryCaretaker - зберігач історії

PlaylistHistoryCaretaker керує колекцією Memento-об'єктів. Він додає та повертає останній збережений стан. Caretaker визначає, чи можлива операція «Скасувати».

4. PlaylistHistoryService - координатор відновлення

PlaylistHistoryService поєднує Originator і Caretaker. Він надає методи SaveState(), Undo() та CanUndo(). Сервіс приховує складність механізму від контролерів і бізнес-логіки.

5. PlaylistsController - клієнт патерну

PlaylistsController викликає збереження стану перед операціями Rename/Delete.

Також він викликає Undo на основі PlaylistHistoryService.

6. TracksController - додатковий клієнт, що змінює стан плейліста

TracksController впливає на вміст плейліста (Add/Delete треків). Оскільки ці дії змінюють стан плейліста, вони також можуть зберігатися у історії.

```
Services > Playlists > C# PlaylistMemento.cs
1  using System.Collections.Generic;
2
3  namespace MusicPlayerWeb.Services.Playlists;
4
5  public class PlaylistMemento
6  {
7      public PlaylistMemento(Guid playlistId, string title, IReadOnlyList<Guid> trackIds)
8      {
9          PlaylistId = playlistId;
10         Title = title;
11         TrackIds = trackIds;
12         SavedAtUtc = DateTime.UtcNow;
13     }
14
15     public Guid PlaylistId { get; }
16     public string Title { get; }
17     public IReadOnlyList<Guid> TrackIds { get; }
18     public DateTime SavedAtUtc { get; }
19 }
```

Рис-6. Services-Playlists-PlaylistMemento.cs

Services > Playlists > C# PlaylistOriginator.cs

```
1  using System;
2  using System.Collections.Generic;
3  using System.Linq;
4  using MusicPlayerWeb.Models;
5
6  namespace MusicPlayerWeb.Services.Playlists;
7
8  public class PlaylistOriginator
9  {
10     public Guid PlaylistId { get; private set; }
11     public string Title { get; private set; } = string.Empty;
12     public IReadOnlyList<Guid> TrackIds { get; private set; } = Array.Empty<Guid>();
13
14     public void Load(Playlist playlist, IEnumerable<Track> tracks)
15     {
16         PlaylistId = playlist.Id;
17         Title = playlist.Title;
18         TrackIds = tracks.Select(t => t.Id).ToArray();
19     }
20
21     public PlaylistMemento Save()
22     {
23         return new PlaylistMemento(PlaylistId, Title, TrackIds.ToArray());
24     }
25
26     public void Restore(PlaylistMemento memento)
27     {
28         PlaylistId = memento.PlaylistId;
29         Title = memento.Title;
30         TrackIds = memento.TrackIds.ToArray();
31     }
32 }
```

Рис-7. Services-Playlists-PlaylistOriginator.cs

Services > Playlists > C# PlaylistHistoryCaretaker.cs

```
1  using System.Collections.Generic;
2
3  namespace MusicPlayerWeb.Services.Playlists;
4
5  public class PlaylistHistoryCaretaker
6  {
7      private readonly Stack<PlaylistMemento> _history = new();
8      private readonly object _sync = new();
9
10     public void Push(PlaylistMemento memento)
11     {
12         lock (_sync)
13         {
14             _history.Push(memento);
15         }
16     }
17
18     public PlaylistMemento? Pop()
19     {
20         lock (_sync)
21         {
22             if (_history.Count == 0)
23                 return null;
24             return _history.Pop();
25         }
26     }
27
28     public bool HasHistory
29     {
30         get
31         {
32             lock (_sync)
33             {
34                 return _history.Count > 0;
35             }
36         }
37     }
38 }
39 }
```

Рис-8. Services-Playlists-PlaylistHistoryCaretaker.cs

```

1  using System;
2  using System.Linq;
3  using Microsoft.Extensions.Caching.Memory;
4  using MusicPlayerWeb.Services;
5
6  namespace MusicPlayerWeb.Services.Playlists;
7
8  public class PlaylistHistoryService : IPlaylistHistoryService
9  {
10     private readonly IPlaylistService _playlists;
11     private readonly IMemoryCache _cache;
12     private static readonly TimeSpan HistoryLifetime = TimeSpan.FromMinutes(30);
13
14     public PlaylistHistoryService(IPlaylistService playlists, IMemoryCache cache)
15     {
16         _playlists = playlists;
17         _cache = cache;
18     }
19
20     public bool Backup(string username, Guid playlistId)
21     {
22         var playlist = _playlists.GetById(username, playlistId);
23         if (playlist == null)
24             return false;
25
26         var tracks = _playlists.GetTracks(username, playlistId).ToList();
27         var originator = new PlaylistOriginator();
28         originator.Load(playlist, tracks);
29
30         var caretaker = _cache.GetOrCreate(BuildKey(username, playlistId), entry =>
31         {
32             entry.SlidingExpiration = HistoryLifetime;
33             return new PlaylistHistoryCaretaker();
34         });
35
36         if (caretaker == null)
37             return false;
38
39         caretaker.Push(originator.Save());
40         return true;
41     }
42
43     public bool Undo(string username, Guid playlistId)
44     {
45         if (!_cache.TryGetValue(BuildKey(username, playlistId), out PlaylistHistoryCaretaker? caretaker) || caretaker == null)
46             return false;
47
48         var snapshot = caretaker.Pop();
49         if (snapshot == null)
50             return false;
51
52         var originator = new PlaylistOriginator();
53         originator.Restore(snapshot);
54         return ApplyState(username, originator);
55     }
56

```

```

56
57 public bool HasHistory(string username, Guid playlistId)
58 {
59     if (!_cache.TryGetValue(BuildKey(username, playlistId), out PlaylistHistoryCaretaker? caretaker) || caretaker == null)
60         return false;
61
62     return caretaker.HasHistory;
63 }
64
65 private bool ApplyState(string username, PlaylistOriginator originator)
66 {
67     var restored = _playlists.Restore(username, originator.PlaylistId, originator.Title);
68     if (!restored)
69         return false;
70
71     var existing = _playlists.GetTracks(username, originator.PlaylistId)
72         .Select(t => t.Id)
73         .ToList();
74
75     foreach (var trackId in existing)
76     {
77         _playlists.RemoveTrack(username, originator.PlaylistId, trackId);
78     }
79
80     foreach (var trackId in originator.TrackIds)
81     {
82         _playlists.AddTrack(username, originator.PlaylistId, trackId);
83     }
84
85     return true;
86 }
87
88 private static string BuildKey(string username, Guid playlistId)
89 {
90     return $"playlist-history:{username}:{playlistId}";
91 }
92 }
93

```

Рис-9-10. Services-Playlists-PlaylistHistoryService.cs

Services > Playlists > C# IPlaylistHistoryService.cs

```

1 namespace MusicPlayerWeb.Services.Playlists;
2
3 public interface IPlaylistHistoryService
4 {
5     bool Backup(string username, Guid playlistId);
6     bool Undo(string username, Guid playlistId);
7     bool HasHistory(string username, Guid playlistId);
8 }
9

```

Рис-11. Services-Playlists-IPlaylistHistoryService.cs

Controllers > C# PlaylistsController.cs

```
1 using Microsoft.AspNetCore.Authorization;
2 using Microsoft.AspNetCore.Mvc;
3 using MusicPlayerWeb.Models;
4 using MusicPlayerWeb.Services;
5 using MusicPlayerWeb.Services.Playlists;
6 using System.Linq;
7
8 namespace MusicPlayerWeb.Controllers;
9
10 [Authorize]
11 public class PlaylistsController : Controller
12 {
13     private readonly IPlaylistService _playlists;
14     private readonly IPlaylistHistoryService _history;
15
16     public PlaylistsController(IPlaylistService playlists, IPlaylistHistoryService history)
17     {
18         _playlists = playlists;
19         _history = history;
20     }
21
22     [HttpPost]
23     public IActionResult Create(string title)
24     {
25         var username = User.Identity!.Name!;
26
27         if (string.IsNullOrEmpty(title))
28         {
29             TempData["Error"] = "Назва плейліста є обов'язковою.";
30             return RedirectToAction("Index", "Home");
31         }
32
33         var playlist = _playlists.Create(username, title);
34         return RedirectToAction("Index", "Home", new { playlistId = playlist.Id });
35     }
36
37     [HttpPost]
38     public IActionResult Rename(Guid playlistId, string newTitle)
39     {
40         var username = User.Identity!.Name!;
41
42         if (string.IsNullOrEmpty(newTitle))
43         {
44             TempData["Error"] = "Нова назва не може бути порожньою.";
45             return RedirectToAction("Index", "Home", new { playlistId });
46         }
47
48         if (!_history.Backup(username, playlistId))
49         {
50             TempData["Error"] = "Плейліст не знайдено для створення знімка стану.";
51             return RedirectToAction("Index", "Home", new { playlistId });
52         }
53
54         var ok = _playlists.Rename(username, playlistId, newTitle);
55     }
56 }
```

```

55
56     if (!ok)
57         TempData["Error"] = "Плейліст не знайдено.";
58
59     return RedirectToAction("Index", "Home", new { playlistId });
60 }
61
62 [HttpPost]
63 public IActionResult Delete(Guid playlistId)
64 {
65     var username = User.Identity!.Name!;
66
67     if (!_history.Backup(username, playlistId))
68     {
69         TempData["Error"] = "Плейліст не знайдено.";
70         return RedirectToAction("Index", "Home");
71     }
72
73     var ok = _playlists.Delete(username, playlistId);
74
75     if (!ok)
76     {
77         TempData["Error"] = "Плейліст не знайдено.";
78         return RedirectToAction("Index", "Home");
79     }
80
81     TempData["Info"] = "Плейліст видалено. Натисни \"Скасувати\", щоб повернути його.";
82     return RedirectToAction("Index", "Home", new { playlistId });
83 }
84
85 [HttpPost]
86 public IActionResult Undo(Guid playlistId)
87 {
88     var username = User.Identity!.Name!;
89     if (!_history.Undo(username, playlistId))
90     {
91         TempData["Error"] = "Немає попереднього стану для відновлення.";
92     }
93     else
94     {
95         TempData["Info"] = "Стан плейліста відновлено.";
96     }
97
98     return RedirectToAction("Index", "Home", new { playlistId });
99 }
100
101 [HttpGet]

```

Рис-12-13. Controllers-PlaylistController.cs

Controllers > C# TracksController.cs

```
1 using Microsoft.AspNetCore.Authorization;
2 using Microsoft.AspNetCore.Mvc;
3 using MusicPlayerWeb.Services;
4 using MusicPlayerWeb.Services.Playlists;
5
6 namespace MusicPlayerWeb.Controllers;
7
8 [Authorize]
9 public class TracksController : Controller
10 {
11     private readonly IWebHostEnvironment _env;
12     private readonly ITrackService _tracks;
13     private readonly IPlaylistService _playlists;
14     private readonly IPlaylistHistoryService _history;
15
16     public TracksController(IWebHostEnvironment env, ITrackService tracks, IPlaylistService playlists, IPlaylistHistoryService history)
17     {
18         _env = env;
19         _tracks = tracks;
20         _playlists = playlists;
21         _history = history;
22     }
23
24     [HttpPost]
25     public async Task<ActionResult> Add(string title, IFormFile file, Guid playlistId)
26     {
27         var username = User.Identity!.Name!;
28
29         if (playlistId == Guid.Empty)
30         {
31             TempData["Error"] = "Вибери плейліст.";
32             return RedirectToAction("Index", "Home");
33         }
34
35         if (string.IsNullOrWhiteSpace(title))
36         {
37             TempData["Error"] = "Назва обов'язкова.";
38             return RedirectToAction("Index", "Home", new { playlistId });
39         }
40
41         if (file == null || file.Length == 0)
42         {
43             TempData["Error"] = "Файл не вибраний.";
44             return RedirectToAction("Index", "Home", new { playlistId });
45         }
46
47         if (!_history.Backup(username, playlistId))
48         {
49             TempData["Error"] = "Плейліст не знайдено.";
50             return RedirectToAction("Index", "Home", new { playlistId });
51         }
52
53         var uploads = Path.Combine(_env.WebRootPath, "uploads");
54         Directory.CreateDirectory(uploads);
55     }
56 }
```

```

56     var safe = $"{Guid.NewGuid()}_{Path.GetExtension(file.FileName)}";
57     var path = Path.Combine(uploads, safe);
58
59     await using (var stream = System.IO.File.Create(path))
60     {
61         await file.CopyToAsync(stream);
62     }
63
64     var rel = $"/uploads/{safe}";
65     var track = _tracks.AddForUser(username, title, safe, rel);
66
67     var added = _playlists.AddTrack(username, playlistId, track.Id);
68     if (!added)
69     {
70         TempData["Error"] = "Не вдалося додати трек до плейліста.";
71     }
72
73     return RedirectToAction("Index", "Home", new { playlistId });
74 }
75
76 [HttpPost]
77 public IActionResult Delete(Guid id, Guid playlistId)
78 {
79     var username = User.Identity!.Name!;
80     if (!_history.Backup(username, playlistId))
81     {
82         TempData["Error"] = "Плейліст не знайдено.";
83         return RedirectToAction("Index", "Home", new { playlistId });
84     }
85
86     var removed = _playlists.RemoveTrack(username, playlistId, id);
87     if (!removed)
88     {
89         TempData["Error"] = "Трек не знайдено в плейлісті.";
90     }
91     return RedirectToAction("Index", "Home", new { playlistId });
92 }

```

Рис-14-15. Controllers-TrackController.cs

Views > Home > @ Index.cshtml

```
1  @model MusicPlayerWeb.Models.PlaylistPageViewModel
2
3  <h2>Музичний програвач</h2>
4
5  @if (TempData["Error"] != null)
6  {
7      <div class="alert">@TempData["Error"]</div>
8  }
9  @if (TempData["Info"] != null)
10 {
11     <div class="alert info">@TempData["Info"]</div>
12 }
13
14 <div class="layout">
15
16     <aside class="sidebar card-panel">
17         <h3>Мої плейлісти</h3>
18
19         @foreach (var p in Model.Playlists)
20         {
21             var active = Model.SelectedPlaylistId == p.Id ? "active" : "";
22             <div class="playlist-item @active">
23                 <div class="playlist-link">
24                     <a asp-controller="Home"
25                        asp-action="Index"
26                        asp-route-playlistId="@p.Id">@p.Title</a>
27                 </div>
28                 <form asp-controller="Playlists" asp-action="Rename" method="post" class="inline-form">
29                     <input type="hidden" name="playlistId" value="@p.Id" />
30                     <input type="text" name="newTitle" value="@p.Title" required />
31                     <button title="Перейменувати">Перейменувати</button>
32                 </form>
33                 <form asp-controller="Playlists" asp-action="Delete" method="post" class="inline-form">
34                     <input type="hidden" name="playlistId" value="@p.Id" />
35                     <button class="danger" title="Видалити">✖</button>
36                 </form>
37             </div>
38         }
39
40         <h4>Новий плейліст</h4>
41         <form asp-controller="Playlists" asp-action="Create" method="post" class="new-playlist-form">
42             <input type="text" name="title" required />
43             <button>Створити</button>
44         </form>
45     </aside>
46
```

```

47 <section class="main">
48
49     @if (!Model.SelectedPlaylistId.HasValue)
50     {
51         <p>Вибери плейліст.</p>
52         return;
53     }
54
55     <h3>Плейліст: @Model.SelectedPlaylistTitle</h3>
56     @if (Model.SelectedPlaylistId.HasValue)
57     {
58         <div class="playlist-undo">
59             <form asp-controller="Playlists" asp-action="Undo" method="post" class="inline-form">
60                 <input type="hidden" name="playlistId" value="@Model.SelectedPlaylistId" />
61                 @if (Model.CanUndoPlaylistChanges)
62                 {
63                     <button type="submit">↶ Скасувати останню дію</button>
64                 }
65                 else
66                 {
67                     <button type="submit" disabled title="Немає попереднього стану">↶ Скасувати останню дію</button>
68                 }
69             </form>
70         </div>
71     }
72
73     <section class="uploader card-panel">
74         <form asp-controller="Tracks" asp-action="Add" enctype="multipart/form-data" method="post">
75
76             <label>До якого плейліста додати?</label>
77             <select name="playlistId" required>
78                 @foreach (var p in Model.Playlists)
79                 {
80                     <option value="@p.Id" selected="@((p.Id == Model.SelectedPlaylistId))">
81                         @p.Title
82                     </option>
83                 }
84             </select>
85
86             <label>Назва треку</label>
87             <input name="title" required />
88
89             <label>Файл</label>
90             <input type="file" name="file" accept="audio/*" required />
91
92             <button type="submit">Додати трек</button>
93         </form>
94     </section>
95

```

```

96     <section class="list card-panel">
97         <h3>Треки</h3>
98
99         @if (!Model.Tracks.Any())
100         {
101             <p>У цьому плейлісті ще немає треків.</p>
102         }
103         else
104         {
105             <table class="tracks">
106                 <thead>
107                     <tr><th>Назва</th><th>Дія</th></tr>
108                 </thead>
109                 <tbody>
110                     @foreach (var t in Model.Tracks)
111                     {
112                         <tr data-url="@t.RelativeUrl" data-title="@t.Title">
113                             <td>@t.Title</td>
114                             <td>
115                                 <button class="playThis">>></button>
116
117                                 <form asp-controller="Tracks" asp-action="Delete" method="post" style="display:inline">
118                                     <input type="hidden" name="id" value="@t.Id" />
119                                     <input type="hidden" name="playlistId" value="@Model.SelectedPlaylistId" />
120                                     <button class="danger">Видалити</button>
121                                 </form>
122                             </td>
123                         </tr>
124                     }
125                 </tbody>
126             </table>
127         }
128     </section>
129
130     <section class="player card-panel">
131         <audio id="player" controls preload="metadata" style="width:100%">
132             <source id="playerSource" src="" type="audio/mpeg" />
133             Тер аудіо не підтримується.
134         </audio>
135         <div id="nowPlaying"></div>
136         <div class="controls">
137             <button id="btnPlay">>> Відтворити</button>
138             <button id="btnPause">⏸ Пауза</button>
139             <button id="btnStop" class="danger">■ Стоп</button>
140             <button id="btnBack">⏮ -10с</button>
141             <button id="btnFwd">⏭ +10с</button>
142             <button id="btnShuffle">🔀 Shuffle</button>
143             <button id="btnRepeat">🔁 Repeat</button>
144         </div>
145
146         <div>
147             <section class="equalizer">
148                 <h4>Еквалайзер</h4>
149                 <label for="eqBass">Низькі частоти</label>
150                 <input type="range" id="eqBass" min="-15" max="15" value="0" />
151                 <label for="eqTreble">Високі частоти</label>
152                 <input type="range" id="eqTreble" min="-15" max="15" value="0" />
153             </section>
154         </div>
155
156     </div>
157
158     <script src="~/js/player.js"></script>
159

```

Рис-16-19. Views-Home-Index.cshtml'

Реалізувати не менше 3-х класів відповідно до обраної теми.

Services > C# PlayerStateService.cs

```
1  using MusicPlayerWeb.Models;
2
3  namespace MusicPlayerWeb.Services;
4
5  public class PlayerStateService
6  {
7      private readonly object _sync = new();
8      private PlayerStateSnapshot _snapshot = PlayerStateSnapshot.Empty;
9
10     public PlayerStateSnapshot Snapshot
11     {
12         get
13         {
14             lock (_sync)
15             {
16                 return _snapshot;
17             }
18         }
19     }
20
21     public void Play(string title, string url)
22     {
23         lock (_sync)
24         {
25             _snapshot = PlayerStateSnapshot.Create(title, url, PlayerPlaybackState.Playing);
26         }
27     }
28
29     public void Pause()
30     {
31         lock (_sync)
32         {
33             if (!_snapshot.HasTrack)
34                 return;
35
36             _snapshot = PlayerStateSnapshot.Create(_snapshot.Title, _snapshot.Url, PlayerPlaybackState.Paused);
37         }
38     }
39
40     public void Stop()
41     {
42         lock (_sync)
43         {
44             _snapshot = PlayerStateSnapshot.Create(null, null, PlayerPlaybackState.Stopped);
45         }
46     }
47 }
```

Рис-20. Services – PlayerStateService.cs

Services > Commands > C# PlayerCommandInvoker.cs

```
1  using MusicPlayerWeb.Models;
2
3  namespace MusicPlayerWeb.Services.Commands;
4
5  public class PlayerCommandInvoker
6  {
7      private const int MaxHistory = 20;
8      private readonly List<PlayerCommandHistoryItem> _history = new();
9
10     public IReadOnlyCollection<PlayerCommandHistoryItem> History
11     {
12         get
13         {
14             lock (_history)
15             {
16                 return _history.ToArray();
17             }
18         }
19     }
20
21     public void Execute(IPlayerCommand command, string username)
22     {
23         command.Execute();
24         AddToHistory(command.Name, username);
25     }
26
27     private void AddToHistory(string commandName, string username)
28     {
29         lock (_history)
30         {
31             _history.Add(new PlayerCommandHistoryItem
32             {
33                 CommandName = commandName,
34                 ExecutedBy = username,
35                 ExecutedAtUtc = DateTime.UtcNow
36             });
37
38             if (_history.Count > MaxHistory)
39             {
40                 _history.RemoveRange(0, _history.Count - MaxHistory);
41             }
42         }
43     }
44 }
```

Рис-21. Services – Commands - PlayerCommandInvoker.cs

```
1  using System;
2  using System.Linq;
3  using Microsoft.Extensions.Caching.Memory;
4  using MusicPlayerWeb.Services;
5
6  namespace MusicPlayerWeb.Services.Playlists;
7
8  public class PlaylistHistoryService : IPlaylistHistoryService
9  {
10     private readonly IPlaylistService _playlists;
11     private readonly IMemoryCache _cache;
12     private static readonly TimeSpan HistoryLifetime = TimeSpan.FromMinutes(30);
13
14     public PlaylistHistoryService(IPlaylistService playlists, IMemoryCache cache)
15     {
16         _playlists = playlists;
17         _cache = cache;
18     }
19
20     public bool Backup(string username, Guid playlistId)
21     {
22         var playlist = _playlists.GetById(username, playlistId);
23         if (playlist == null)
24             return false;
25
26         var tracks = _playlists.GetTracks(username, playlistId).ToList();
27         var originator = new PlaylistOriginator();
28         originator.Load(playlist, tracks);
29
30         var caretaker = _cache.GetOrCreate(BuildKey(username, playlistId), entry =>
31         {
32             entry.SlidingExpiration = HistoryLifetime;
33             return new PlaylistHistoryCaretaker();
34         });
35
36         if (caretaker == null)
37             return false;
38
39         caretaker.Push(originator.Save());
40         return true;
41     }
42
43     public bool Undo(string username, Guid playlistId)
44     {
45         if (!_cache.TryGetValue(BuildKey(username, playlistId), out PlaylistHistoryCaretaker? caretaker) || caretaker == null)
46             return false;
47
48         var snapshot = caretaker.Pop();
49         if (snapshot == null)
50             return false;
51
52         var originator = new PlaylistOriginator();
53         originator.Restore(snapshot);
54         return ApplyState(username, originator);
55     }
56 }
```

```

57 public bool HasHistory(string username, Guid playlistId)
58 {
59     if (!_cache.TryGetValue(BuildKey(username, playlistId), out PlaylistHistoryCaretaker? caretaker) || caretaker == null)
60         return false;
61
62     return caretaker.HasHistory;
63 }
64
65 private bool ApplyState(string username, PlaylistOriginator originator)
66 {
67     var restored = _playlists.Restore(username, originator.PlaylistId, originator.Title);
68     if (!restored)
69         return false;
70
71     var existing = _playlists.GetTracks(username, originator.PlaylistId)
72         .Select(t => t.Id)
73         .ToList();
74
75     foreach (var trackId in existing)
76     {
77         _playlists.RemoveTrack(username, originator.PlaylistId, trackId);
78     }
79
80     foreach (var trackId in originator.TrackIds)
81     {
82         _playlists.AddTrack(username, originator.PlaylistId, trackId);
83     }
84
85     return true;
86 }
87
88 private static string BuildKey(string username, Guid playlistId)
89 {
90     return $"playlist-history:{username}:{playlistId}";
91 }
92 }
93

```

Рис-22-23. Services – Playlists – PlaylistHistoryService.cs

Висновок:

У ході виконання лабораторної роботи я опрацювала шаблони проектування Abstract Factory, Factory Method, Memento, Observer та Decorator та розібралася з їхнім призначенням, структурою та особливостями використання. На прикладі свого веб-застосунку «Музичний програвач» я реалізувала патерн Memento для скасування змін у плейлістах, що дозволило відновлювати попередній стан без втручання у внутрішню логіку та структуру даних. Під час роботи я переконалася, що використання шаблонів робить систему гнучкішою, легшою для супроводу та розширення, а також допомагає уникати дублювання коду. Виконання цієї лабораторної роботи дало мені можливість не лише зрозуміти теорію патернів, але й застосувати їх на практиці у власному проєкті, що значно поглибило мої навички та впевненість у розробці програмного забезпечення.

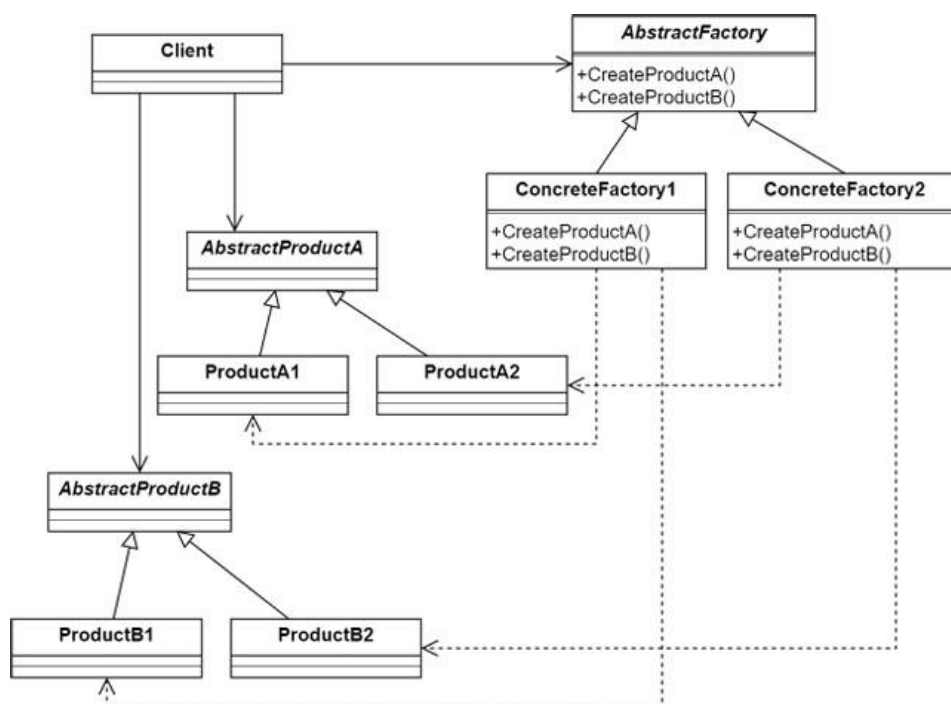
Відповіді на контрольні запитання:

1. Призначення шаблону «Абстрактна фабрика»

Шаблон «Абстрактна фабрика» (Abstract Factory) призначений для створення сімейств пов'язаних об'єктів без прив'язки до конкретних класів.

Він дозволяє клієнту працювати лише з інтерфейсами, не знаючи, які саме об'єкти створюються.

2. Структура шаблону «Абстрактна фабрика»



3. Класи та їх взаємодія в «Абстрактній фабриці»

AbstractFactory - оголошує інтерфейс для створення абстрактних продуктів.

ConcreteFactory - реалізує методи створення конкретних продуктів.

AbstractProduct - описує інтерфейс продуктів.

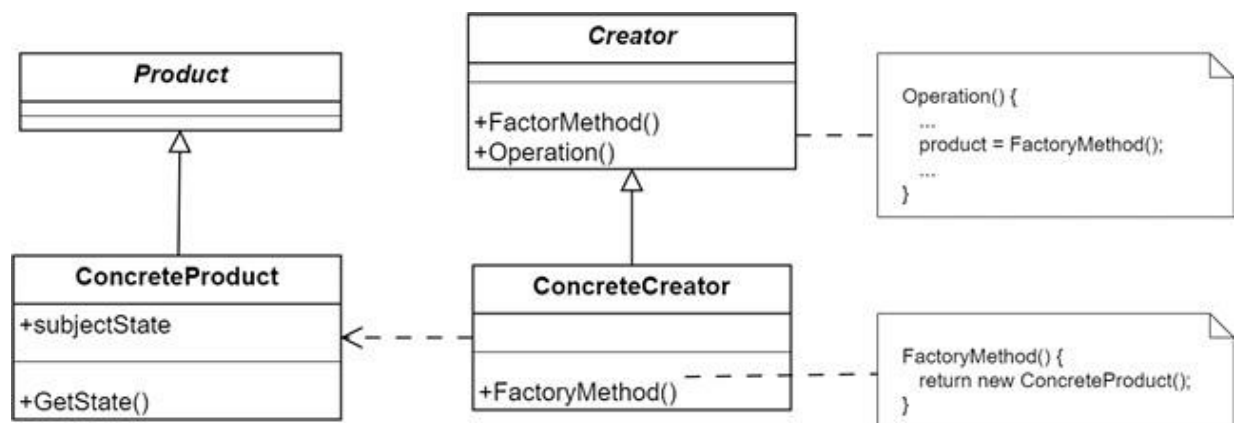
ConcreteProduct - реалізує конкретний продукт.

Client - лише з інтерфейсами **AbstractFactory** та **AbstractProduct**, не знаючи про їх реалізацію.

4. Призначення шаблону «Фабричний метод»

Шаблон «Фабричний метод» (Factory Method) визначає інтерфейс для створення об'єкта, але дозволяє підкласам вирішувати, який саме клас створювати. Він делегує створення об'єктів підкласам.

5. Структура шаблону «Фабричний метод»



6. Класи та взаємодія у «Фабричному методі»

Product - спільний інтерфейс для всіх продуктів.

ConcreteProduct - конкретна реалізація продукту.

Creator - оголошує фабричний метод

FactoryMethod().

ConcreteCreator - перевизначає FactoryMethod() для створення конкретного продукту.

Client - використовує об'єкт через базовий інтерфейс Product.

7. Відмінність між «Абстрактною фабрикою» та «Фабричним методом»

«Фабричний метод» визначає інтерфейс для створення одного типу об'єкта, але дозволяє підкласам вирішувати, який саме клас створювати. Тобто він зосереджується на створенні окремого продукту, надаючи механізм розширення без зміни існуючого коду. Основна ідея полягає в тому, щоб делегувати створення об'єкта підкласам.

«Абстрактна фабрика», своєю чергою, оперує на вищому рівні - вона створює цілі сімейства пов'язаних об'єктів, які повинні використовуватися разом. Наприклад, фабрика може створювати набір інтерфейсних елементів (кнопки, меню, поля вводу), що належать до певного стилю (Windows, macOS тощо).

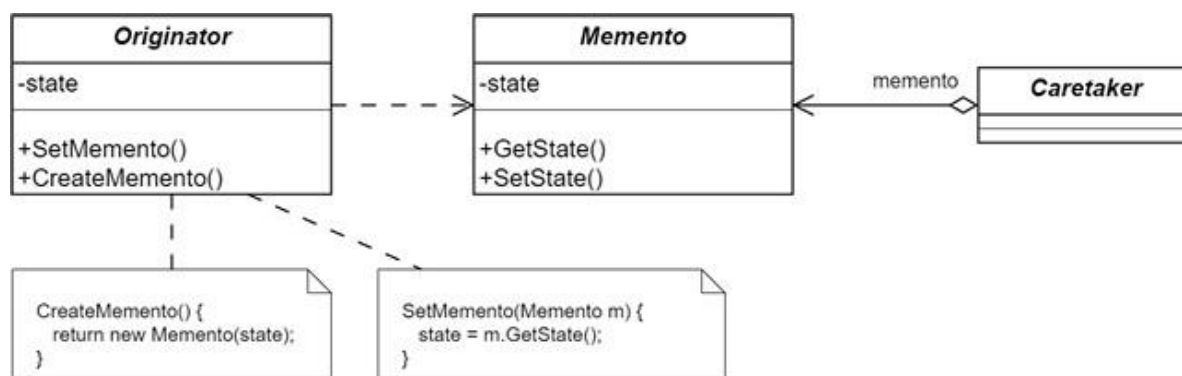
Отже, «Фабричний метод» вирішує, як створити один об'єкт, а «Абстрактна фабрика» - як створити набір сумісних об'єктів. Часто «Абстрактна фабрика» реалізується через кілька фабричних методів усередині себе.

8. Призначення шаблону «Знімок» (Memento)

Шаблон «Знімок» використовується для збереження і відновлення попереднього стану об'єкта без порушення інкапсуляції.

Він дозволяє «відкотити» об'єкт до попереднього стану (наприклад, у функції «Undo»).

9. Структура шаблону «Знімок»



10. Класи та взаємодія у «Знімку»

Originator - об'єкт, стан якого потрібно зберегти. Створює **Memento** і відновлює стан із нього.

Memento - зберігає внутрішній стан Originator.

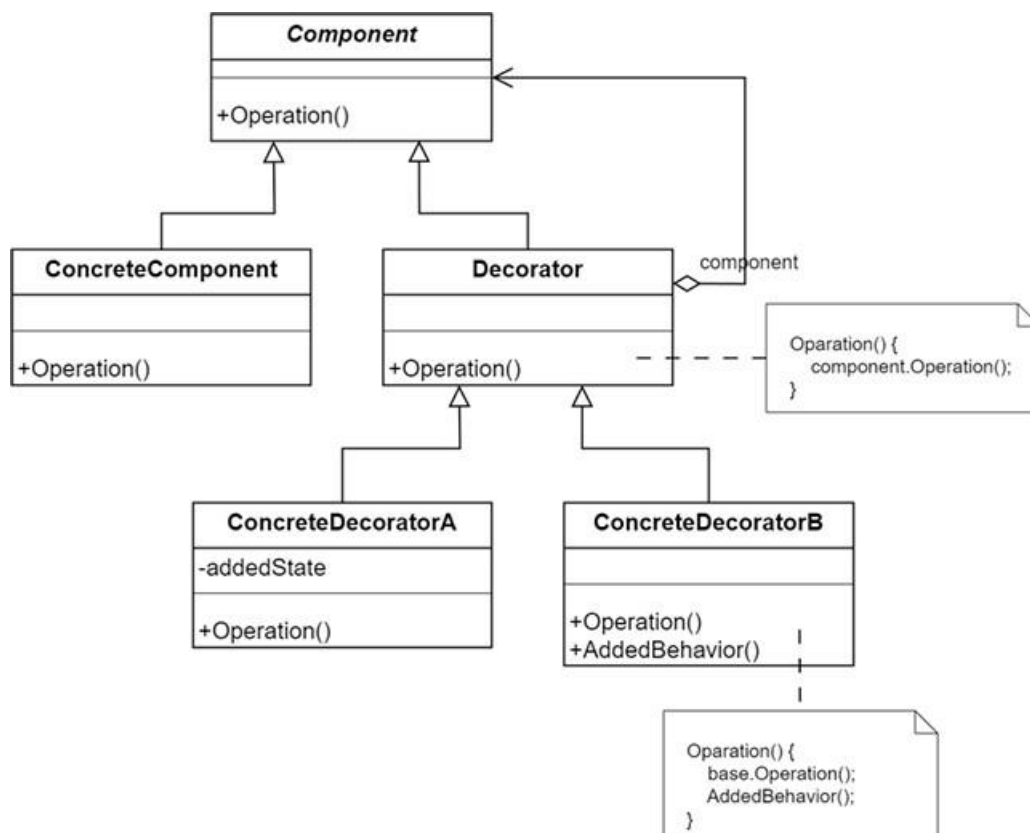
Caretaker - відповідає за збереження та відновлення Memento, але не змінює його вміст.

11. Призначення шаблону «Декоратор»

Шаблон «Декоратор» (Decorator) дозволяє динамічно додавати нову поведінку або функціональність об'єктам без зміни їх коду.

Він «обгортає» об'єкт у додатковий клас, який розширює його можливості.

12. Структура шаблону «Декоратор»



13. Класи та взаємодія у «Декораторі»

Component - базовий інтерфейс або клас.

ConcreteComponent - основний об'єкт, який може бути «обгорнутий».

Decorator - базовий клас, який реалізує інтерфейс Component і містить посилання на нього.

ConcreteDecorator - додає нову поведінку до об'єкта перед або після виклику базових методів.

14. Обмеження використання шаблону «Декоратор»

Велика кількість маленьких об'єктів ускладнює відлагодження.

Може бути важко зрозуміти послідовність викликів при вкладених декораторах.

Не підходить, коли потрібно змінювати поведінку всіх об'єктів класу одразу (краще використати наслідування)